



Today's agenda

↳ Adapter design Pattern

↳ Facade design Pattern



AlgoPrep



* Structural design Pattern

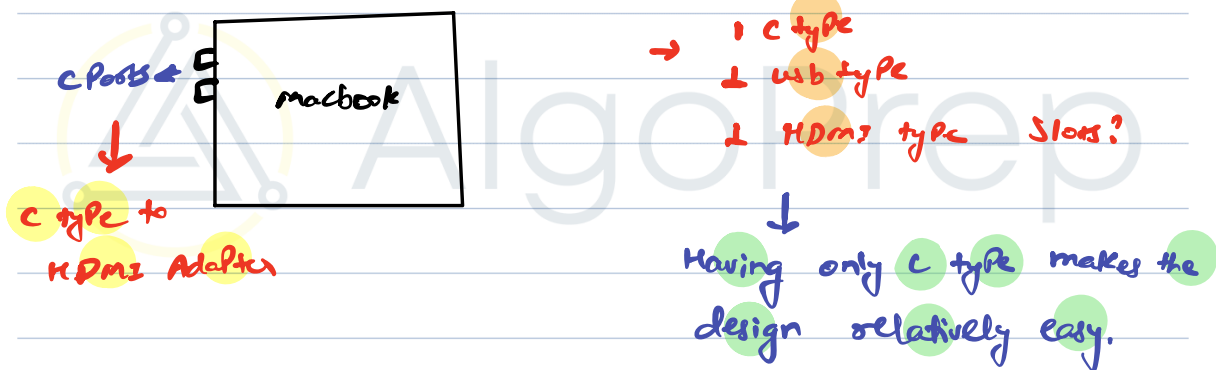
↳ How to structure our codebase.

↳ classes/Interface decisions.

↳ attributes within a class.

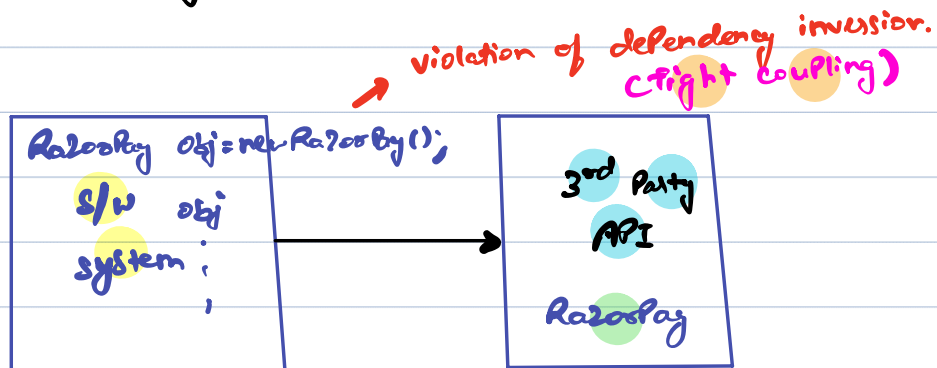
① Adapter design Pattern

↳ Power Adapter → Convert one form to another

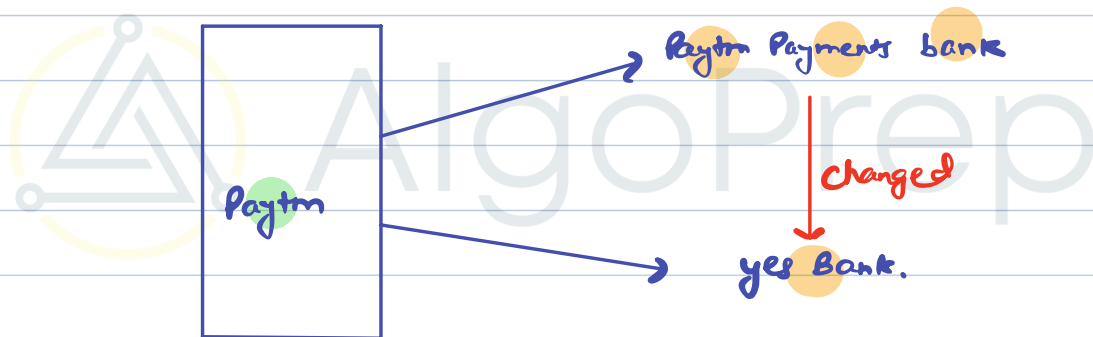




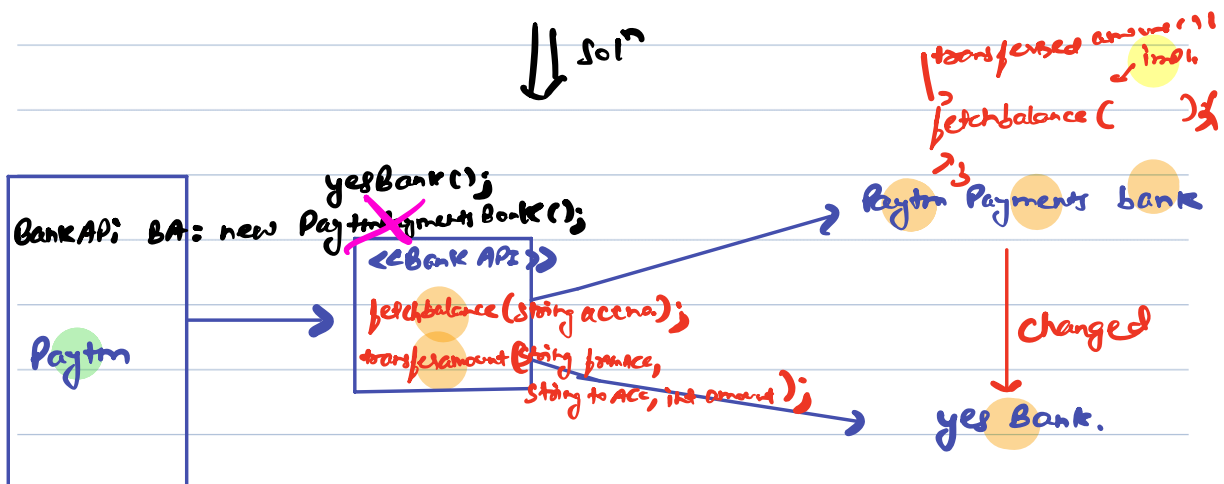
↳ Adapter design Pattern

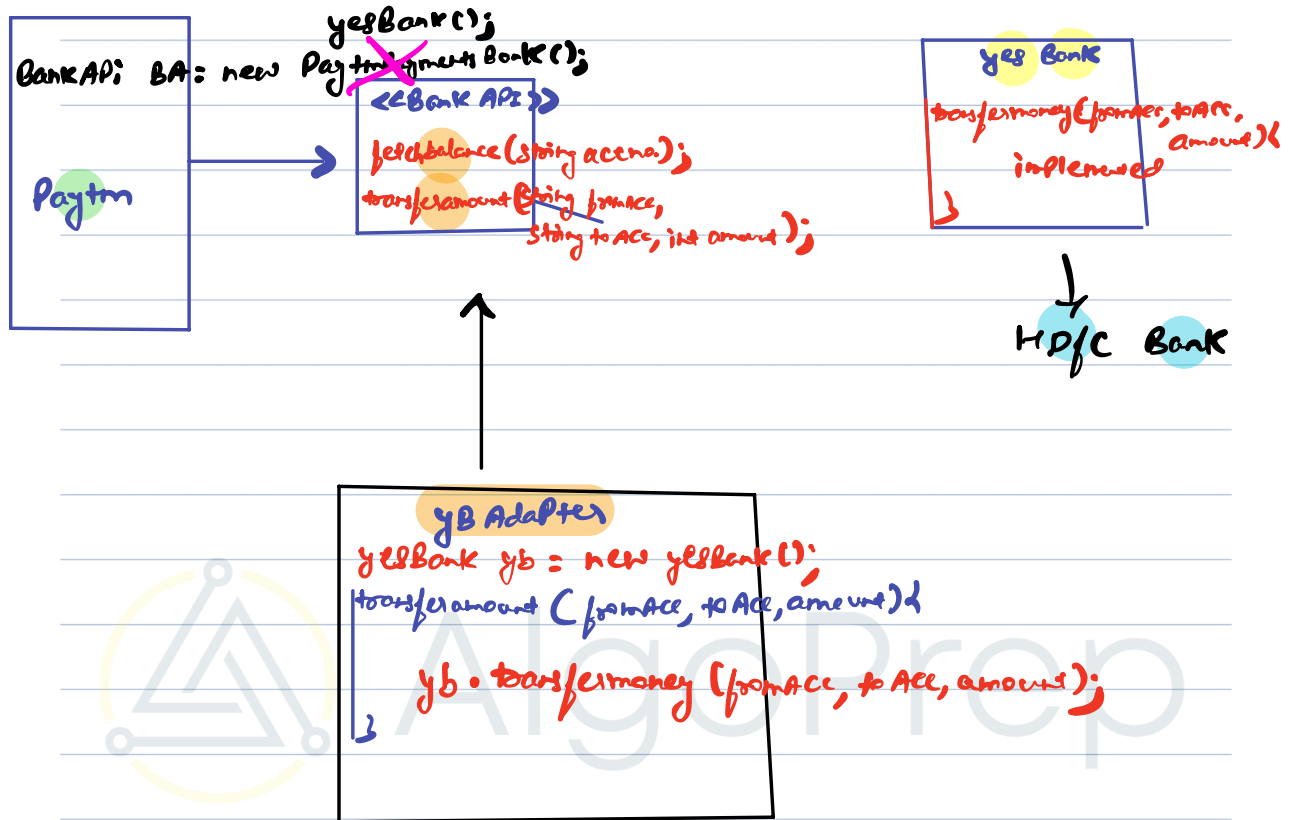


→ Paytm Problem Statement.



↓ solⁿ







→ How to use Adapters

↳ 1. Whenever we are connecting to a 3rd Party APIs, create an interface with all the required methods.

2. Create an adapter class which is going to act as a bridge between yours and 3rd Party service provider.

→ Calendly?

↳ 3:00pm 4:00pm
 ↳ 3:15 3:30 3:45 4:00

↳ name
↳ email

3rd Party APIs.

↳ google meet / zoom
↳ calendar

⋮

→ Cal.com → A competitor of Calendly with open sourced codebase.



* Facade design Pattern

↳ outer face

→

Blinkit

```
Inventory Ims;  
DeliveryPartner DP;  
notifyService NS;  
orderPlaced() {  
    Ims.update();  
    DP.assignPartner();  
    NS.sendOrderConfirmation();  
}
```

→ SRP violation.

→ long code.

```
orderCanceled() {
```

```
}
```

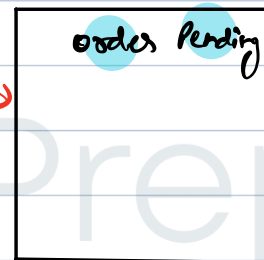
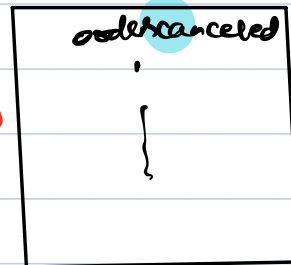
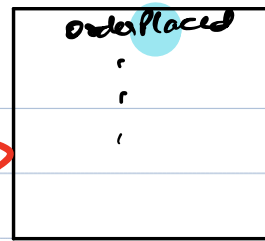
```
pendingOrder() {
```

```
}
```

3rd level entity



Blinkit



AlgoPrep